

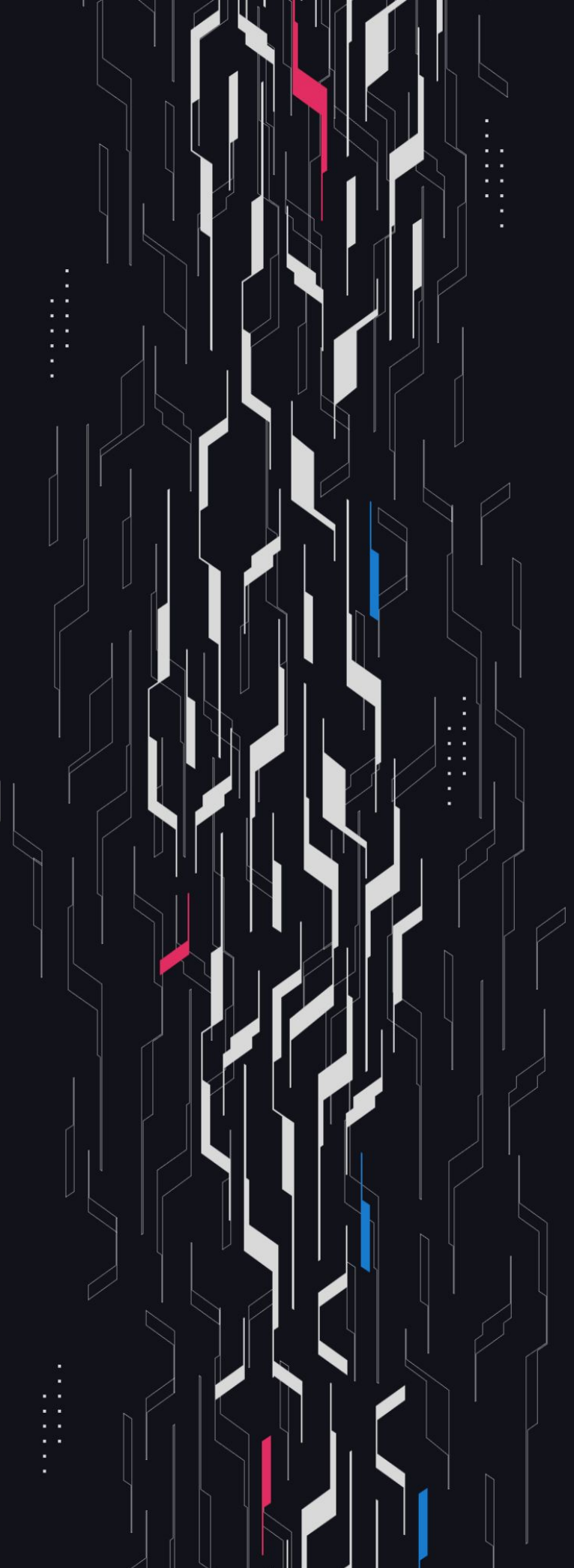
GA GUARDIAN

USDT0

Hedera HTS and EVM

Security Assessment

March 11th, 2026



Summary

Audit Firm Guardian

Prepared By Minato Namakazi, Osman Ozdemir

Client Firm USDT0

Final Report Date 03/12/2026

Audit Summary

USDT0 engaged Guardian to review the security of its Hedera HTS and EVM. From the 11/24/2025 to the 01/07/2026, a team of 2 auditors reviewed the source code in scope. All findings have been recorded in the following report.

Confidence Ranking

Given the minimal High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

Note: Fixes to the findings uncovered in the remediation review have not been reviewed by Guardian.



Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>



Code coverage & PoC test suite:

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 28

About Guardian 29

Project Overview

Project Summary

Project Name	USDT0
Language	Solidity
Codebase	https://github.com/hiero-ledger/hiero-consensus-node/
Commit(s)	b71f1a623f0144d486adaa99619421e86ea05153

Audit Summary

Delivery Date	03/12/2026
Audit Methodology	Static Analysis, Manual Review, Test Suite, Contract Fuzzing

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	1	0	0	0	0	1
● High	0	0	0	0	0	0
● Medium	8	0	3	1	0	3
● Low	4	0	2	0	0	2
● Info	11	1	0	10	0	0

Audit Scope & Methodology

hedera-node/hedera-token-service/src/main/java/com/hedera/node/app/service/token/*
hedera-node/hedera-token-service-impl/src/main/java/com/hedera/node/app/service/token/impl/*
hedera-node/hedera-smart-contract-service/src/main/java/com/hedera/node/app/service/contract/*
hedera-node/hedera-smart-contract-service-impl/src/main/java/com/hedera/node/app/service/contract/impl/*
hedera-node/hedera-util-service-impl/src/main/java/com/hedera/node/app/service/util/impl/*
hedera-node/hedera-util-service/src/main/java/com/hedera/node/app/service/util/*

Language	files	blank	comment	code
Java	588	7014	17085	50784
SUM:	588	7014	17085	50784

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Findings & Resolutions

ID	Title	Category	Severity	Status
C-01	Consensus Topics Authorization Bypass	Validation	● Critical	Resolved
M-01	Fungible Token Wipe Corrupts NFT Metadata	Logical Error	● Medium	False Positive
M-02	Zero Wipe Corrupt numberPositiveBalances Counter	Logical Error	● Medium	Resolved
M-03	Airdrop Handler Fails To Resolve Aliases	Logical Error	● Medium	False Positive
M-04	Lambda Hooks Can Create Hidden Zero Valued Slots	DoS	● Medium	Resolved
M-05	Mismatch Allows Hook To Exhaust Resources	DoS	● Medium	Resolved
M-06	LambdaSStoreHandler Undercharges Slot Clearings	Gas Griefing	● Medium	
M-07	HTS 0x167 Dispatch Runs Before Gas Enforcement	DoS	● Medium	Acknowledged
L-01	Airdrop Handler Bypasses Token Association Limit	Validation	● Low	False Positive
L-02	Gas Under Charged For HBAR Lazy Account Creation	Informational	● Low	Resolved
L-03	Untrusted Hook Execution Bumps Owner Eth Nonce	DoS	● Low	False Positive
I-01	Incorrect Code Comment	Typo	● Info	Acknowledged

Findings & Resolutions

ID	Title	Category	Severity	Status
I-02	Misleading Proto Comment	Documentation	● Info	Acknowledged
I-03	Possible Unbounded Memory Cache	Suggestion	● Info	Acknowledged

C-01 | Consensus Topics Authorization Bypass

Category	Severity	Location	Status
Validation	● Critical	ConsensusUpdateTopicHandler.java: 143	Resolved

Description

In the `ConsensusUpdateTopicHandler` authorization logic for topic updates, extending the expiry is the only update operation permitted without an admin key. Last year, Hedera released HIP-991, which introduced custom fees and `fee_schedule_key`.

`onlyExtendsExpiry()` does not check `op.hasCustomFees()` , `op.hasFeeExemptKeyList()`, or `op.hasFeeScheduleKey()`. So a transaction that sets `expirationTime` and modifies `customFees` or `feeExemptKeyList` or `feeScheduleKey` will still satisfy `onlyExtendsExpiry(op) == true`. And `preHandle()` will return early and never call the required authorization checks.

Even if a topic was created with no custom fees and without a `fee_schedule_key` (so fees should be impossible/immutable by design), an attacker can still send `ConsensusUpdateTopic` with `expirationTime` set and a non-empty `custom_fees` field.

From that point on, `ConsensusSubmitMessageHandler` will charge those `custom_fees` whenever someone sends a message to the topic, even though the topic was originally not allowed to have fees at all. This can be applied on every application currently using the Hedera Consensus Service.

Recommendation

An attacker can set fees so large that messages to all topics always fail. No one will be able to submit any `ConsensusSubmitMessage` to any topic/application, causing the entire Consensus Service applications networks to halt

Or they can charge fees of all topics of all applications to themselves by setting themselves as the `fee_collector_account_id` in the arbitrary `FixedCustomFees`, and make all `ConsensusSubmitMessage` to all topics on the network charge a large fixed amount of HBAR/HTS tokens from the message sender to the exploiter

Mitigate by update `onlyExtendsExpiry()` and `wantsToMutateNonExpiryField()` to include `op.hasFeeScheduleKey()`, `op.hasFeeExemptKeyList()`, and `oop.hasCustomFees()`

Resolution

USDT0: Fixed in this releases tag
<https://github.com/hiero-ledger/hiero-consensus-node/releases/tag/v0.68.4>

M-01 | Fungible Token Wipe Corrupts NFT Metadata

Category	Severity	Location	Status
Logical Error	● Medium	TokenAccountWipeHandler.java: 191	False Positive

Description

In Hedera, a wipeKey is a special administrative key on a token that authorizes forced removal of tokens from an account without that account’s consent

So for any fungible token T

If T has a wipeKey, whoever controls that key can call the wipe operation to subtract some or all units of T from a target account

Wiping removes some or all of that account’s balance of T

There is a bug that can arbitrarily corrupt NFT related metadata for an account during a fungible token wipe

because the handler always uses the serialNumbers list even when the token is fungible and no NFTs are involved

```
// Always build an NFT serial list, even for fungible tokens
final var nftSerialNums = new ArrayList<>(new LinkedHashSet<>(op.serialNumbers()));
// Always decrement the account's NFT count by that list size
updatedAcctBuilder.numberOwnedNfts(acct.numberOwnedNfts() - nftSerialNums.size());
```

This numberOwnedNfts update happens unconditionally, outside if (token.tokenType() == TokenType.FUNGIBLE_COMMON)

And in the fungible branch we never validate or use nftSerialNums at all

Recommendation

for fungible tokens, serial numbers must not be provided

Resolution

M-02 | Zero Wipe Corrupt numberPositiveBalances Counter

Category	Severity	Location	Status
Logical Error	● Medium	TokenAccountWipeHandler.java: 194	Resolved

Description

In Hedera, zero unit fungible wipes are allowed per HIP-564, so a wipe with `amount = 0` is considered valid.

In `TokenAccountWipeHandler.handle()` for `FUNGIBLE_COMMON`, because `fungibleWipeCount >= 0` is allowed and `validateSemantics()` does not require the account token balance to be greater than zero, a transaction with `amount = 0` and an existing token relation with balance `0` is valid. This can happen when an account is associated to a token but has never received any units, or when it previously held tokens but later transferred or wiped all of them and the relation remained with zero balance.

`numberPositiveBalances` represents the number of tokens with a positive balance associated with an account.

If the fungible balance is zero, a zero-amount wipe decrements `numberPositiveBalances` even though no positive balance changed.

In that case, `validated.accountTokenRel().balance() - fungibleWipeCount` is `0 - 0 = 0`, so `newAccountBalance == 0` is true, and the code decrements `numberPositiveBalances` even though the original balance was already zero and no positive balance changed.

This is inconsistent with how `numberPositiveBalances` is maintained elsewhere. `BaseTokenHandler.adjustBalance()` only updates the counter when crossing zero based on both `originalBalance` and `newBalance`.

That counter is then used in `TokenServiceApiImpl` to enforce `TRANSACTION_REQUIRES_ZERO_TOKEN_BALANCES`, so if `numberPositiveBalances` is driven down with repeated zero-amount wipes on zero-balance relations, it can reach `0` while some token relations still have a positive balance.

Recommendation

Treat `numberPositiveBalances` exactly like `adjustBalance()` does

Resolution

USDT0: Fixed @ <https://github.com/hiero-ledger/hiero-consensus-node/pull/22580>

M-03 | Airdrop Handler Fails To Resolve Aliases

Category	Severity	Location	Status
Logical Error	● Medium	TokenAirdropHandler.java: 141	False Positive

Description

In the `TokenAirdropHandler` , the `handle` method and its interaction with `CustomFeeAssessmentStep`

The code assumes that the `TokenAirdropTransactionBody` , passed to `assessAndChargeCustomFee` contains fully resolved Account IDs, or that the `CustomFeeAssessmentStep` and underlying stores can correctly handle Aliases (EVM addresses) during fee assessment.

But, `CustomFeeAssessmentStep` performs lookups in `ReadableTokenRelationStore` and `ReadableAccountStore` using the Account IDs provided in the transaction body.

If these IDs are Aliases, the stores, which utilize `WritableKVState` keyed by `EntityIDPair` or `AccountID` will fail to find the records because they expect numeric IDs or the specific key format used in state.

Unlike `CryptoTransferHandler`, which explicitly calls `ensureAndReplaceAliasesInOp` to resolve aliases before assessing custom fees, `TokenAirdropHandler` performs validation , which resolves aliases internally, but then passes the original body containing aliases to the fee assessor.

This bug causes a valid class of transactions to fail, locking users out of airdropping tokens to aliased accounts if those tokens have custom fees

Recommendation

The `TokenAirdropHandler` should resolve aliases in the `convertedOp`, replace them with numeric Account IDs before calling `assessAndChargeCustomFee`

Resolution

USDT0: Notes on Verdict Section

M-04 | Lambda Hooks Can Create Hidden Zero Valued Slots

Category	Severity	Location	Status
DoS	● Medium	HookEvmFrameState.java	Resolved

Description

For normal contracts, the system tracks. The value a storage slot had at the start of the transaction `original` and the value it has at the end `final`.

If a slot didn't exist, its original value is treated as 0, when a transaction finishes, the code compares `original` vs `final` to decide if storage was inserted, removed, or updated, and then charges rent / enforces limits based on that. For regular contract storage, if a slot does not exist and the contract writes zero to it, the code does not create any entry

But Hook contracts use a different storage path. Here, when a slot does not exist and the hook writes zero, the system does create a slot with value 0. If user deploys a contract and registers it as a lambda EVM hook on their account, when the hook runs during a `CryptoTransfer`, and perform many `SSTORE` operations to fresh keys that end with zero values. It will leaves behind a real `SlotValue` entries in `LAMBDA_STORAGE`

These slots are invisible to per account lambda slot counters and rent and expiry walks, because they are not included in the linked list. This causes permanent state bloat and extra disk usage because the slots :

Consume disk and `VirtualMap` capacity, are not reflected in `hookState.numStorageSlots` or `account.numberLambdaStorageSlots`

Are not reachable via the normal linked list rooted at `EvmHookState.firstContractStorageKey`, so hook deletion / expiry won't traverse them.

Recommendation

.
Make hook storage follow the same "zero into empty slot is no-op and zero final means no slot" rules as normal contract storage in both the write path and commit path.

Resolution

USDT0: Fixed @ <https://github.com/hiero-ledger/hiero-consensus-node/pull/22770>

M-05 | Mismatch Allows Hook To Exhaust Resources

Category	Severity	Location	Status
DoS	● Medium	CryptoTransferHandler.java: 308	Resolved

Description

In `CryptoTransferHandler.java` related with the calculation of fees for EVM hooks attached to token transfers

If we compare how the fee is calculated / charged and how the gas limit is enforced during execution

In `CryptoTransferHandler.calculateFees`, the code calculates the `effectiveGasLimit` to charge the user and explicitly clamps the total gas limit requested by the hooks to `maxGasPerSec`

But in `TransferExecutor.java`, when the hooks are actually dispatched for execution, the code uses the raw gas limit specified in the transaction body via `hookCallsFactory`, without applying the same clamping logic or validating that it matches the charged amount

The `HookInvocation` objects are created in `HookCallsFactory.java` directly from the transaction body values

An attacker can construct a `CryptoTransfer` transaction with hooks specifying an extremely large gas limit, with value significantly higher than `maxGasPerSec`

The System will charge the user only for `maxGasPerSec` due to the `Math.min` clamping in `calculateFees` and the execution will dispatch the EVM hook with the fully requested significantly higher gas limit.

This allows attacker to execute computation heavy logic inside a hook that runs far longer than the block gas limit would normally permit, while paying a capped fee.

This can keep stalling the nodes and exhausting network resources cheaply

Recommendation

The execution logic must enforce the same limits used in fee calculation. The `TransferExecutor` or `CryptoTransferValidator` should validate that the total gas limit requested by hooks does not exceed `maxGasPerSec`

Resolution

USDT0: Fixed @ <https://github.com/hiero-ledger/hiero-consensus-node/pull/22797>

M-06 | LambdaSStoreHandler Undercharges Slot Clearings

Category	Severity	Location	Status
Gas Griefing	● Medium	LambdaSStoreHandler.java: 63	

Description

ZERO_INT0_NONZERO_GAS_COST is set to a fixed value of 200 gas in LambdaSStoreHandler. This is intended to mimic the EVM’s gas refund behavior when clearing storage slots.

In the EVM, writing zero to a non-zero storage slot costs 5,000 gas, and a 4,800 gas refund is recorded. However, this refund is capped at one-fifth of the total gas expenditure, and the user must provide the full 5,000 gas upfront.

LambdaSStoreHandler charges only the net 200 gas and behaves as if the entire 4,800 gas refund is always applied, which does not reflect reality in most cases.

Currently, the maximum number of storage slots that can be touched in a single transaction in LambdaSStore defaults to 10 (maxLambdaSStoreUpdates).

In the EVM, clearing 10 storage slots requires 50,000 gas upfront. Although a 48,000 gas refund is tracked, it is capped at 20% of the total gas cost, so the effective refund is limited and the total gas expenditure remains far higher than 200 gas per slot.

In Hedera LambdaSStore, a fixed charge of 2,000 gas is applied for 10 slot updates.

For a fixed cost of 200 gas per slot to be applied correctly, assuming the full 4,800 gas refund per slot and a 20% refund cap, the transaction would need to cost at least 240,000 gas to break even. Any transaction below this threshold that clears 10 storage slots is therefore undercharged compared to EVM semantics.

Attackers can spam state cleanup operations at a fraction of the intended cost, undermining fee fairness and opening a DoS vector that consumes resources much more cheaply than the work performed.

Recommendation

Align effectiveGasCost with EIP-2200/EIP-3529: charge 5,000 gas upfront for non-zero to zero transitions, record a refundable credit separately, and apply the 20 % refund cap at the transaction level. Alternatively, if Hedera intends to deviate from EVM semantics, document the rationale and enforce a higher fixed cost than 200 gas to prevent fee abuse.

Resolution

M-07 | HTS 0x167 Dispatch Runs Before Gas Enforcement

Category	Severity	Location	Status
DoS	● Medium	DispatchForResponseCodeHtsCall.java: 205	Acknowledged

Description

The HTS system contract 0x167 is executed before the EVM frame enforces gas sufficiency

An attacker can forward intentionally tiny gas to HTS guaranteeing `remainingGas < gasRequirement` yet still force the node to perform expensive HTS work of `dispatch()` and only afterwards halt with `INSUFFICIENT_GAS`

Because Gas is checked only after `computeFully()`, In `doExecuteSystemContract()` `computeFully()` executes the call before deciding insufficiency, In `resultOfExecuting()`

For many HTS operations, `dispatch()` is the expensive step and runs first In `DispatchForResponseCodeHtsCall.execute()`

An attacker can craft calldata that maximizes HTS work, with any expensive per transfer, per NFT, per association loops inside HTS operations

Even though the call is guaranteed to end as `INSUFFICIENT_GAS`, the node still runs `dispatch()` and builds records

The node burns CPU and I/O doing work that is guaranteed to be thrown away, full PoC here <https://gist.github.com/GuardianAudits/807dd307791b647ceb1e166ba5a33f70>

Recommendation

In `DispatchForResponseCodeHtsCall.execute()` compute `gasRequirement` first, If `frame.getRemainingGas() < gasRequirement`, return an insufficient gas result without calling `dispatch()`

Resolution

USDT0: Acknowledged.

L-01 | Airdrop Handler Bypasses Token Association Limit

Category	Severity	Location	Status
Validation	● Low	TokenClaimAirdropHandler.java: 152	False Positive

Description

in TokenClaimAirdropHandler

When a user claims a pending airdrop for a token they are not already associated with, the handler automatically associates the account with the token using the associateForFree method.

But unlike TokenAssociateToAccountHandler, this process fails to validate the tokens.maxPerAccount limit defined in the network configuration, EntitiesConfig.limitTokenAssociations and TokensConfig.maxPerAccount

This allows a user to bypass the maximum token association limit imposed on accounts by repeatedly claiming pending airdrops, potentially leading to state bloat and resource exhaustion on a single account, which the limit is specifically designed to prevent.

Recommendation

Update TokenClaimAirdropHandler to perform the same association limit checks as TokenAssociateToAccountHandler, verify that the account's current number of associations plus the new ones being added does not exceed the configured maxPerAccount limit

Resolution

USDT0: This is expected behavior

L-02 | Gas Under Charged For HBAR Lazy Account Creation

Category	Severity	Location	Status
Informational	● Low	ClassicTransfersCall.java: 218	Resolved

Description

minimumTinycentPriceGiven() calculates the tiny cent price for the HTS atomic cryptoTransfer precompile. It charges per HBAR, token, and NFT transfer

It adds baseLazyCreationPrice, but only when a token or NFT transfer credits an alias that does not yet map to an account

But It never checks for aliases in the HBAR TransferList

Therefore minimumTinycentPriceGiven() currently ignores HBAR aliases when building aliasesToLazyCreate, so it never adds baseLazyCreationPrice for HBAR based lazy creations

As a result, each new account created via HBAR alias credits is billed only the basic HBAR transfer cost in gas, instead of the intended CRYPTO_CREATE + CRYPTO_UPDATE cost

Recommendation

.
Make minimumTinycentPriceGiven() also scan the HBAR TransferList for alias AccountIDs that do not resolve and add baseLazyCreationPrice once per such alias, consistent with token/NFT processing

Resolution

USDT0: Fixed @ <https://github.com/hiero-ledger/hiero-consensus-node/releases/tag/v0.69.0>

L-03 | Untrusted Hook Execution Bumps Owner Eth Nonce

Category	Severity	Location	Status
DoS	● Low	AbstractProxyEvmAccount.java: 75	False Positive

Description

In the ProxyEvmHook class which handles account nonces. Because the hook proxy shares the underlying state identity with the Hook Owner, execution of CREATE and CREATE2 ,

Within a hook triggers an increment of the Hook Owner native ethereumNonce on the ledger. This allows an attacker to arbitrarily advance an account nonce without the owner signature, leading to a denial of service for valid Ethereum transactions from that account. In ProxyEvmHook, it passes the Hook Owner's Account ID to the parent constructor, Then AbstractProxyEvmAccount,

Implements getNonce() and setNonce() by directly reading from and writing to the ledger state of the bound account ID. When the EVM executes CREATE or CREATE2 opcodes, the Besu EVM implementation unconditionally calls account.incrementNonce() on the executing account to prevent contract address collisions

So when a hook executes a creation opcode, it unintentionally invokes setNonce on the owner real ledger account.

Any user which has installed a lambda EVM hook on their account, any third party can repeatedly trigger it by sending top level CryptoTransfer transactions that include preTx / prePost hook fields on transfers involving the victim.

Which drives ProxyEvmHook execution without needing the victim signature

After each call, the attacker can push the victim ethereumNonce arbitrarily far forward, causing all lowernonce EthereumTransaction operations from that account to fail

Recommendation

Maintain a separate hook local nonce.

Resolution

USDT0: "This is expected behavior given the very peculiar situation of someone installing a hook that uses CREATE/CREATE2 opcodes"

I-01 | Incorrect Code Comment

Category	Severity	Location	Status
Typo	● Info	Hts0x167TranslatorsModule.java: 22	Acknowledged

Description

"Provides the {@link CallTranslator} implementations for the HTS system contract at address 0x16c." comment in the Hts0x167TranslatorsModule file is incorrect as it is for the contract at address 0x167 not 0x16c.

Recommendation

Update the comment: "... HTS system contract at address 0x167."

Resolution

USDT0: Acknowledged.

I-02 | Misleading Proto Comment

Category	Severity	Location	Status
Documentation	● Info	ContractDeleteHandler.java: 70	Acknowledged

Description

Proto comment in `response_code` states that `OBTAINER_REQUIRED` applies “when deleting smart contract that has crypto balance”, but `ContractDeleteHandler` unconditionally enforces an obtainer even when the contract’s HBAR balance is zero. The proto wording misleads integrators into thinking a zero-HBAR contract could omit an obtainer.

Recommendation

Either do not require obtainer when deleted contract has zero HBAR balance, or update the comment to prevent misunderstandings.

Resolution

USDT0: Acknowledged.

I-03 | Possible Unbounded Memory Cache

Category	Severity	Location	Status
Suggestion	● Info	InnerTxnCache.java: 24-26	Acknowledged

Description

In InnerTxnCache

```
transactionsCache = Caffeine.newBuilder()
    .expireAfterWrite(INNER_TXN_CACHE_TIMEOUT, TimeUnit.SECONDS)
    .build(transactionBytes -> transactionParser.parse(transactionBytes, configuration));
```

It caches entries keyed by raw Bytes that come from the transaction atomicBatchOrThrow().transactions() which is user controlled input

The cache has no maximumSize() and no maximumWeight()

The current time based eviction expireAfterWrite(15s) does not prevent memory blowups, it just bounds them by

"whatever can be inserted in 15 seconds"

So an attacker could drive heap usage extremely high by submitting many atomic batches containing many unique inner transaction byte blobs

Even if each entry only lives ~15 seconds, the heap growth during those 15 seconds could cause gc thrashing

Recommendation

.
Add hard bounds to the cache in transactionsCache

And consider adding the maxNumberOfTransactions() check in preHandle() too, so we fail fast and don't parse / cache work we'll reject anyway

Resolution

USDT0: Stress-tested in testnet, low priority

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	Permanent State Bloat Via Net Zero Storage Write	Logical Error	● Medium	False Positive
I-01	NFT Treasury Update Desynchronization	Logical Error	● Info	Acknowledged

M-01 | Permanent State Bloat Via Net Zero Storage Write

Category	Severity	Location	Status
Logical Error	● Medium	HookEvmFrameState.java / WritableEvmHookStore.java	False Positive

Description

The fix in `setStorageValue()` only blocks the single step case `SSTORE(0 -> 0)` on a fresh key, but there is still exploitable net zero variant. A hook can do `SSTORE(K, 1) SSTORE ((K, 0)` for a fresh key in the same hook execution. The first write creates a real `SlotValue` entry immediately via the put path.

```
writableEvmHookStore.updateStorage(slotKey, slotValue)
```

But does not link it or increment counters at commit, `persistChanges()` converts the final 32 byte zero word to `Bytes.EMPTY` removing the marker and calls `updateStorage()`.

`applyStorageMutations()` classifies this as `ZERO_INT0_EMPTY_SLOT` and the switch default does nothing, so it never removes the already created entry.

That leaves behind real `LAMBDA_STORAGE` entries with value 0, `prev/next = EMPTY`, not in the linked list and not counted in `hookState.numStorageSlots` and `account.numberEvmHookStorageSlots` enables unbounded permanent state bloat and let's hook deletion/expiry miss these slots again, same end state as the previous bug

Recommendation

Add this case in `applyStorageMutations()`

```
case ZERO_INT0_EMPTY_SLOT -> {
    storage.remove(new EvmHookSlotKey(hookId, minimalKey(update.key())));
    storage.remove(slot.key());
    yield firstKey;
}
```

Resolution

USDT0: False Positive.

I-01 | NFT Treasury Update Desynchronization

Category	Severity	Location	Status
Logical Error	● Info	TokenUpdateHandler.sol	Acknowledged

Description

There is an issue in how the `numberPositiveBalances` counter is updated when an NFT changes its treasury account. because of a contradictory check in the code, the newly assigned treasury receives the NFTs, but its `numberPositiveBalances` counter fails to increase.

When the treasury changes, the system automatically moves all the NFTs held by the old treasury into the new treasury before moving the NFTs, the system enforces this rule the new treasury account must currently have a balance of 0 for this token.

```
validateTrue(newTreasuryRel.balance() == 0, TRANSACTION_REQUIRES_ZERO_TOKEN_BALANCES);
```

After the check, the system calls `changeOwnerToNewTreasury()` to actually move the NFTs and update the `numberPositiveBalances` counter but looking at how it calculates the new counter for the receiving new treasury.

```
final var toRelBalance = toTreasuryRel.balance(); // we know this is 0 because of the check above
// later
final var newToPositiveBalancesCount =
    toRelBalance > 0 ? toTreasury.numberPositiveBalances() + 1 :
toTreasury.numberPositiveBalances();
```

Because `toRelBalance` is forced to be 0, the condition `toRelBalance > 0` is always false, therefore, the new treasury account receives all the NFTs, but `toTreasury.numberPositiveBalances() + 1` is never executed.

The account internal counter becomes desynchronized from the actual assets it holds.

Recommendation

Correct the ternary operator logic in `changeOwnerToNewTreasury()`.

Resolution

USDT0: Acknowledged.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>